

Mamdani Fuzzy Control System for a Line-Following Robot Based on STM32F411

Kevin Adrian Lara Hernandez, Orlando Contreras Reyes, Roberto Lopez Rodriguez¹

Department of Electronic Engineering,
Universidad Autónoma de Aguascalientes, Aguascalientes, México
464376@edu.uaa.mx, 348390@edu.uaa.mx, 234512@edu.uaa.mx

Abstract. This work presents the design and implementation of a Mamdani-type fuzzy control system for a line-following robot. The system uses two reflective sensors (12-bit ADC) as inputs and generates speed commands for two DC motors via pulse-width modulation (PWM). The rule base covers nine combinations of sensor states (black, gray, white); inference is performed using the min-max operator and defuzzification via numerically integrated centroid. The firmware was developed in C on the STM32F411CEU6 microcontroller using the STMicroelectronics HAL, with DMA for simultaneous acquisition of both ADC channels.

Keywords: Fuzzy logic · Mamdani · Centroid · Line follower · STM32F411 · PWM · DMA

1 Introduction

Line-following robots are a classic platform for evaluating real-time control strategies. Fuzzy logic is especially well-suited in this context because optical sensors do not produce perfect binary readings; in transitions between black and white there exists a region of uncertainty that fuzzy sets model naturally [1].

The Mamdani controller [2] allows expert knowledge to be expressed as linguistic rules of the form “*If the left sensor is black and the right sensor is white, then the left motor runs fast and the right motor runs slow*”, which simplifies system tuning without requiring an explicit mathematical model of the robot.

This report describes the complete firmware architecture: membership functions, rule base, inference engine, and analytical/numerical defuzzifier, as well as the peripheral configuration of the STM32F411 (TIM1, ADC1 with DMA, and GPIO).

2 Hardware Platform

2.1 Microcontroller

The system is implemented on an STM32F411CEU6 module (*Black-Pill*), operating at 100 MHz via an external 25 MHz crystal with a PLL multiplier (M=12,

$N=96$, $P=2$). Its most relevant features for this application are: a 12-bit ADC with scan mode and external trigger, four DMA2 channels, and the advanced timer TIM1 with complementary outputs and ADC trigger generation.

2.2 Sensors

Two infrared reflective sensors are connected to PA0 (right sensor) and PA1 (left sensor). Readings are stored in the `carro.Sensor[]` array via circular DMA, triggered by TIM1 channel 3 (*toggle* mode) at the same frequency as the motor PWM (20 kHz), guaranteeing one sample per PWM cycle.

2.3 Actuators

Two DC motors are controlled by an H-bridge. Direction signals (AIN1/AIN2 for the right motor, BIN1/BIN2 for the left) are set at startup in forward mode, while speed is regulated by TIM1 channels CH1 and CH2 in PWM mode. The counter period is 4999 counts, corresponding to $f_{PWM} = 100\text{ MHz}/5\,000 = 20\text{ kHz}$.

2.4 Robot photograph

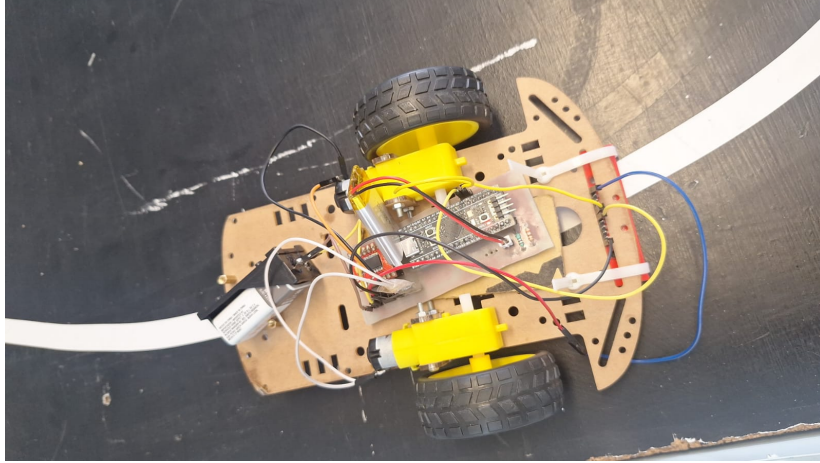


Fig. 1: Line-following robot prototype based on STM32F411CEU6.

3 Fuzzy Logic System Design

The fuzzy system has the following structure:

- **Inputs:** ADC reading of the left sensor (`left_raw`) and the right sensor (`right_raw`).
- **Outputs:** speed percentage for the left motor (LM) and right motor (RM), in the range $[0, 100]$.
- **Inference method:** Mamdani with minimum (AND) and maximum (OR) operators.
- **Defuzzification:** centroid (center of gravity).

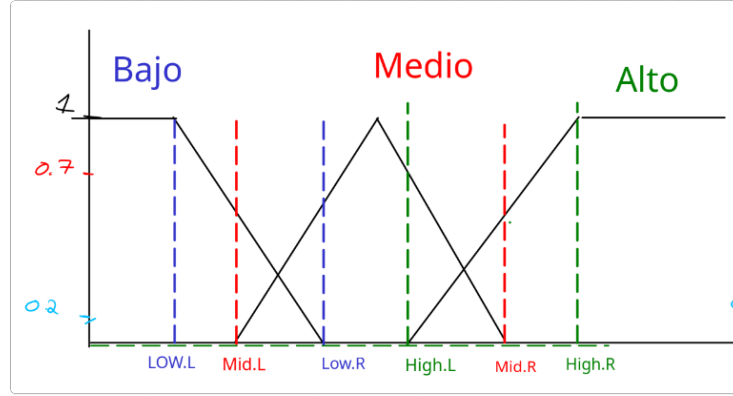


Fig. 2: Motor membership functions.

3.1 Input Fuzzy Sets (Sensors)

Each sensor is classified into three linguistic sets defined over the ADC output range (0–4 095 counts):

Table 1: Input fuzzy set parameters.

Set	L	C	R	Type
White	3 500	—	3 600	Left trapezoid
Gray	3 600	3 760	3 900	Triangular
Black	3 900	—	4 020	Right trapezoid

The membership functions are:

$$\mu_{\text{White}}(x) = \begin{cases} 1 & x \leq 3500 \\ \frac{3600 - x}{3600 - 3500} & 3500 < x < 3600 \\ 0 & x \geq 3600 \end{cases} \quad (1)$$

$$\mu_{\text{Gray}}(x) = \begin{cases} 0 & x \leq 3600 \\ \frac{x - 3600}{3760 - 3600} & 3600 < x \leq 3760 \\ \frac{3900 - x}{3900 - 3760} & 3760 < x < 3900 \\ 0 & x \geq 3900 \end{cases} \quad (2)$$

$$\mu_{\text{Black}}(x) = \begin{cases} 0 & x \leq 3900 \\ \frac{x - 3900}{4020 - 3900} & 3900 < x < 4020 \\ 1 & x \geq 4020 \end{cases} \quad (3)$$

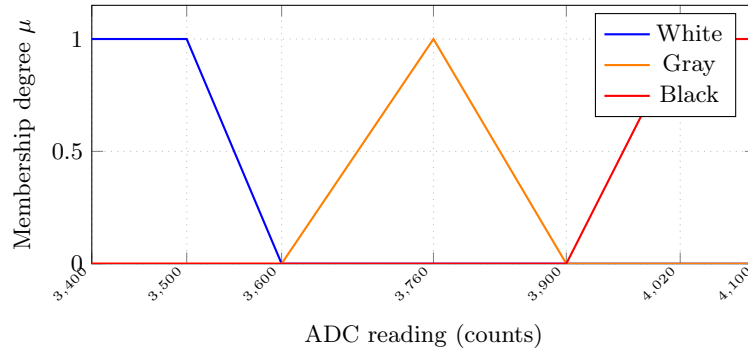


Fig. 3: Input sensor membership functions.

3.2 Output Fuzzy Sets (Motors)

Outputs LM and RM share the same universe of discourse $[0, 100]$ (speed %) with three sets:

Table 2: Output fuzzy set parameters.

Set	L	C	R	Type
Low	0	—	50	Left trapezoid
Mid	25	50	75	Triangular
High	50	—	100	Right trapezoid

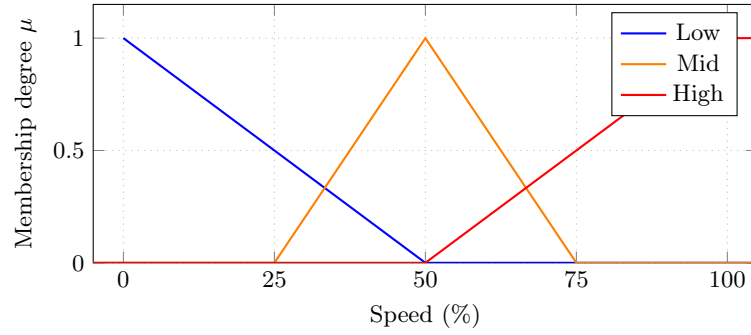


Fig. 4: Output membership functions (motors LM and RM).

4 Mamdani Rule Base

Table 3 shows the nine rules that make up the controller’s knowledge base. Each rule takes the form:

$$\text{IF } L \in A_L \text{ AND } R \in A_R \Rightarrow \text{LM} \in B_{\text{LM}}, \text{RM} \in B_{\text{RM}}$$

where L and R are the left and right sensor readings respectively, A_* are input sets and B_* are output sets. The *weight* parameter scales the rule firing strength before aggregation.

Table 3: Mamdani rule base (L = left sensor, R = right sensor).

#	Sensor L	Sensor R	LM (left)	RM (right)	Weight
1	White	White	High	High	1.0
2	Gray	White	High	Mid	2.0
3	White	Gray	Mid	High	2.0
4	Black	White	High	Low	2.0
5	White	Black	Low	High	2.0
6	Gray	Gray	Mid	Mid	1.0
7	Black	Gray	High	Low	1.0
8	Gray	Black	Low	High	1.0
9	Black	Black	Low	High	1.0

5 Inference Engine

5.1 Fuzzification

For each sensor, the three membership degrees are computed using the functions defined in equations (1)–(3). The implementation in `FuzzyLogic.c` uses the generic functions `Trapz()` and `Triang()`:

```

1 float Trapz(float x, FuzzyArgs args, uint8_t type)
2 {
3     if(type == TRAPZ_LOW){
4         if(x <= args.L) return 1.0f;
5         if(x < args.R) return (args.R - x)/(args.R - args.L)
6     ;
7         return 0.0f;
8     }
9     if(type == TRAPZ_HIGH){
10        if(x <= args.L) return 0.0f;
11        if(x < args.R) return (x - args.L)/(args.R - args.L)
12    ;
13        return 1.0f;
14    }
15 }
16
17 float Triang(float x, FuzzyArgs args) {
18     if(x <= args.L) return 0.0f;
19     if(x < args.C && x > args.L) return (x-args.L)/(args.C-
20     args.L);
21     if(x < args.R && x > args.C) return (args.R-x)/(args.R-
22     args.C);
23     return 0.0f;
24 }

```

Listing 1.1: Generic membership functions (FuzzyLogic.c).

5.2 Rule Evaluation

The `Mamdani_Infer()` function iterates over the rule base, computes each rule's firing strength as $\alpha_i = w_i \cdot \min(\mu_L, \mu_R)$, and aggregates per motor using the maximum operator:

$$y_{\text{LOW}} = \max_i \{\alpha_i \mid \text{consequent}(i) = \text{LOW}\} \quad (4)$$

and analogously for the MID and HIGH sets.

5.3 Numerical Centroid Defuzzification

The `Centroide()` function implements the discrete integral:

$$u^* = \frac{\sum_{x=0}^{100} x \cdot \mu_{\text{agg}}(x)}{\sum_{x=0}^{100} \mu_{\text{agg}}(x)} \quad (5)$$

where $\mu_{\text{agg}}(x) = \max(\hat{\mu}_{\text{LOW}}, \hat{\mu}_{\text{MID}}, \hat{\mu}_{\text{HIGH}})$ with $\hat{\mu}_k(x) = \min(y_k, \mu_k(x))$. The integration step is $\Delta x = 0.1$, yielding 1001 points over the universe of discourse.

```

1 float Centroide(float y_low, float y_mid, float y_high)
2 {
3     float num = 0.0f, den = 0.0f;
4     for(float x = 0.0f; x <= 100.0f; x += 0.1f){
5         float low  = fminf(y_low,
6                             FuzzyMembership(x, Low,  TRAPZ_LOW
7     ));
8         float mid  = fminf(y_mid,
9                             FuzzyMembership(x, Mid,  TRIANG));
10        float high = fminf(y_high,
11                             FuzzyMembership(x, High,
12     TRAPZ_HIGH));
13        float mu    = fmaxf(low, fmaxf(mid, high));
14        num += x * mu;
15        den += mu;
16    }
17    if(den == 0.0f) return 0.0f;
18    return num / den;

```

Listing 1.2: Numerical centroid defuzzification (Centroide.c).

6 Analytical Area Computation (Centroid Module)

In addition to the numerical centroid, the `Centroide.c` module implements analytical area and centroid functions for each region of the aggregated output function. This was designed as a computationally more efficient alternative.

6.1 Initialization (Centroide_Init)

During initialization, intersections between adjacent sets are precomputed:

$$Y_1 = \frac{R_{\text{Low}} \cdot C_{\text{Mid}} - L_{\text{Low}} \cdot L_{\text{Mid}}}{(C_{\text{Mid}} - L_{\text{Mid}}) + (R_{\text{Low}} - L_{\text{Low}})} \quad (6)$$

$$Y_2 = \frac{R_{\text{High}} \cdot R_{\text{Mid}} - L_{\text{High}} \cdot C_{\text{Mid}}}{(R_{\text{Mid}} - C_{\text{Mid}}) + (R_{\text{High}} - L_{\text{High}})} \quad (7)$$

6.2 Area Regions

Five area functions are defined: `Area_Low`, `Area_Mid`, `Area_High`, `Area_InterLm` (Low–Mid intersection), and `Area_InterMh` (Mid–High intersection). Each returns a `DefuzzArgs` structure with numerator ($\sum A_i C_i$) and denominator ($\sum A_i$) to apply the composite centroid theorem:

$$u^* = \frac{\sum_i A_i \cdot C_i}{\sum_i A_i} \quad (8)$$

7 STM32F411 Peripheral Configuration

7.1 System Clock

Table 4: STM32F411 PLL configuration.

Parameter Value	
Source	HSE 25 MHz
PLLM	12
PLLN	96
PLLQ	2
SYSCLK	100 MHz
HCLK	100 MHz
APB1	50 MHz
APB2	100 MHz

7.2 TIM1 – PWM and ADC Trigger

TIM1 operates in *up-counting* mode with prescaler 0 and period $ARR = 4999$, yielding $f = 100 \text{ MHz} / 5000 = 20 \text{ kHz}$.

- **CH1 (PA8):** PWM mode 1, right motor.
- **CH2 (PA9):** PWM mode 1, left motor.
- **CH3 (internal):** *toggle* mode, generates TIM1_TRG0 to trigger the ADC by hardware (ADC_EXTERNALTRIGCONV_T1_CC3).

The duty cycle is updated in the main loop:

```

1 __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1,
2   (uint32_t)(cmd.left * 0.5f * PWM_PERIOD / 100.0f));
3 __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_2,
4   (uint32_t)(cmd.right * 0.5f * PWM_PERIOD / 100.0f));

```

Listing 1.3: PWM update in the main loop (`main.c`).

The 0.5 factor limits the maximum duty cycle to 50% of the period, preventing H-bridge overheating at full-speed conditions.

7.3 ADC1 with DMA

ADC1 operates in two-channel scan mode (CH0 → right sensor, CH1 → left sensor), with a rising-edge external trigger from TIM1_CH3. DMA2_Stream0 automatically transfers results to the `carro.Sensor[2]` array in circular mode, ensuring that the main loop always reads the most recent sample without CPU overhead.

7.4 GPIO

H-bridge direction signals are configured as push-pull outputs on port GPIOA:

Table 5: Direction pin assignments.

Pin	Signal	Function
PA2	AIN1	Right motor direction (1)
PA3	AIN2	Right motor direction (2)
PA4	BIN1	Left motor direction (1)
PA5	BIN2	Left motor direction (2)

At startup, AIN1 and BIN1 are set high (forward).

8 Software Architecture

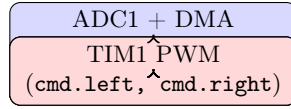


Fig. 5: Fuzzy control system flowchart.

The modular project structure consists of four independent source files:

FuzzyLogic.c/.h Primitive membership functions (**Trapz**, **Triang**), fuzzy operators (AND, OR), and specific membership functions for sensors and motors.
Mamdani.c/.h Rule base, inference engine (**Mamdani_Infer**), and high-level function (**Mamdani_MotorCommand**).
Centroide.c/.h Numerical defuzzifier (**Centroide**) and analytical area functions (**Area_Low**, **Area_Mid**, **Area_High**, **Area_InterLm**, **Area_InterMh**).
main.c/.h HAL configuration, main loop, and **CarroData** structure.

Table 6: Qualitative performance by test scenario.

Scenario	Tracking	Oscillation
Straight line	—	—
Gentle curve	—	—
Sharp curve	—	—
Line loss	—	—

9 Discussion

The nine-rule base exhaustively covers the binary-fuzzy sensor state space. The weights of 2.0 assigned to the heading-correction rules (rows 2–5 of Table 3) increase the influence of those rules on the aggregation engine without altering the Mamdani structure.

The numerical defuzzification cycle with $\Delta x = 0.1$ requires 1 001 iterations per motor, totaling 2 002 evaluations of `FuzzyMembership` per control cycle. At 96 MHz this load is manageable, but the analytical functions in `Centroide.c` could reduce it significantly if computation time becomes a bottleneck.

10 Conclusions

A Mamdani-type fuzzy controller for a line-following robot was successfully designed and implemented on the STM32F411CEU6. The system integrates DMA-based analog acquisition, synchronized PWM generation, and a complete fuzzy inference chain organized into independent, reusable modules.

Future work includes:

1. Validating the analytical centroid functions and replacing the numerical method to reduce computation time.
2. Tuning the input fuzzy set thresholds through online calibration.
3. Adding a controlled braking rule when both sensors detect black for more than N consecutive cycles.

References

1. Zadeh, L.A.: Fuzzy sets. *Information and Control* **8**(3), 338–353 (1965). [https://doi.org/10.1016/S0019-9958\(65\)90241-X](https://doi.org/10.1016/S0019-9958(65)90241-X)
2. Mamdani, E.H., Assilian, S.: An experiment in linguistic synthesis with a fuzzy logic controller. *International Journal of Man-Machine Studies* **7**(1), 1–13 (1975). [https://doi.org/10.1016/S0020-7373\(75\)80002-2](https://doi.org/10.1016/S0020-7373(75)80002-2)
3. Ross, T.J.: *Fuzzy Logic with Engineering Applications*, 3rd edn. Wiley, Chichester (2010).
4. STMicroelectronics: STM32F411xC/E Reference Manual (RM0383). Rev. 3 (2023). https://www.st.com/resource/en/reference_manual/rm0383-stm32f411xce-advanced-armbased-32bit-mcus-stmicroelectronics.pdf